

0.1 Learning Without Rational Expectations

This chapter studies world models in reinforcement learning, learned forecasters of state transitions and rewards used by an agent to plan its actions. In the language of economics a world model is a perceived law of motion that the agent estimates from data and then plans against, an object that economics has studied since at least the 1980s under names such as adaptive learning, fictitious play, and Berk-Nash equilibrium. The subsection that follows briefly catalogues these analogues before the chapter develops the reinforcement-learning line in detail.

0.1.1 A family of non-rational-expectations learning approaches

The first approach is belief-based learning from game theory, in which an agent maintains a representation of opponents' play or of the environment and refines it as new observations arrive. Fictitious play ([Brown, 1951](#)), its smoothed variant ([Fudenberg and Kreps, 1993](#)), and stochastic fictitious play under global convergence conditions ([Hofbauer and Sandholm, 2002](#)) all share with model-based reinforcement learning the basic move of starting from a possibly wrong belief and updating it from data, and they ask the same kinds of convergence question (whether beliefs concentrate, whether they diverge, whether they select among multiple equilibria). The textbook treatments in [Fudenberg and Levine \(1998\)](#) and [Young \(2004\)](#) carry the main results, and the parallels collected below are in that same family rather than formal equivalences.

The second is recursive least squares adaptive learning, developed by [Marcet and Sargent \(1989\)](#) and synthesized in [Evans and Honkapohja \(2001\)](#). In this approach the agent holds a perceived linear law of motion and updates its coefficients from realized data; convergence to a rational-expectations equilibrium holds when an associated ordinary differential equation is locally stable at the fixed point and fails otherwise. The construction is single-rule and on-policy, and it contains no explicit exploration mechanism, which sharply constrains the regions in which it converges.

The third is genetic algorithm learning ([Arifovic, 1994, 1995](#)). A population of binary-encoded decision rules evolves under fitness-proportional selection, crossover, mutation, and an election operator that admits offspring only when they would have outperformed their parents on the previous environment. The augmented genetic algorithm converges

to the rational-expectations equilibrium of the cobweb in both stable and unstable parameter regions, including those in which recursive least squares diverges, with exploration provided by population diversity rather than by a noise term on a single policy.

The fourth is evolutionary heuristic switching (Brock and Hommes, 1997), in which agents choose among a finite set of forecasting heuristics by a logit on past performance with intensity-of-choice parameter β . As β rises from zero the joint dynamics trace a period-doubling cascade from a stable steady state to high-dimensional attractors, and the source of complexity is the discrete selection rule itself rather than the model class or any exploration operator.

The fifth is misspecified Bayesian learning, formalized as Berk-Nash equilibrium by Esponda and Pouzo (2016). The agent’s subjective model class need not contain the truth; beliefs concentrate on parameters minimizing Kullback-Leibler divergence to the data-generating process, and at equilibrium the strategy is optimal given the belief while the belief is best-fit given the strategy. Misspecification is permanent rather than a transient phase as in the four approaches above.

The parallel collected here is architectural rather than an equivalence claim; adaptive learning asks whether beliefs converge while model-based reinforcement learning asks whether a learned transition model improves control, and the success criterion that follows the two literatures is correspondingly different.

The five approaches share a common formal vocabulary, drawn from stochastic approximation and fixed-point analysis, and they address related but distinct convergence questions. Single-agent model-based reinforcement learning, the line this chapter touches on next, asks whether a policy approaches optimality on the true environment as the learned model and the policy are updated together. Multi-agent learning in agent-based models, including Marimon et al. (1990), Krusell and Smith (1998), and Calvano et al. (2020), asks which rational-expectations equilibrium the joint dynamics select when several exist, and whether the selected equilibrium matches the lab-experimental or empirical attractor. Both questions concern the limiting behavior of non-rational-expectations learning dynamics, the machinery is similar, and the objects differ; one is the optimality gap of a single policy, the other is the basin of attraction of a system of interacting learners. The machinery for both traces back to the algorithmic background developed in §??. With this brief catalogue in place, the chapter turns next to the two 1990 origin

papers that opened the model-based line, one from the reinforcement-learning tradition and one from the recurrent-neural-network tradition.

0.2 Two 1990 Origins: Dyna and Differentiable World Models

Two papers published in 1990 introduced what the modern literature now calls a world model. In each, an agent learns a forecaster of its environment from interaction data and uses simulated experience drawn from that forecaster to improve its policy, rather than waiting for real transitions to accumulate. Sutton (1990) arrived at the architecture through dynamic programming and reinforcement learning, framing it as the unification of direct and indirect updates within a single control loop. Schmidhuber (1990) arrived at it through recurrent neural networks, framing it as a way to make the environment itself differentiable for credit assignment. The two papers do not cite one another and use different vocabularies, but the architectural commitment is the same. We treat them in parallel rather than in lineage, since each anticipates a distinct branch of the modern literature.

0.2.1 Sutton’s integrated architecture

Sutton (1990) proposes Dyna as an architecture that interleaves three processes at every real time step. The agent performs a direct Q-learning update from the observed transition (s, a, r, s') , then updates a learned model $\hat{P}(s' | s, a)$ and $\hat{r}(s, a)$ from the same transition, then performs K planning updates by sampling previously visited state-action pairs $(\tilde{s}, \tilde{a})$, querying the model for simulated outcomes $(\tilde{r}, \tilde{s}')$, and applying the same Q-update to those synthetic transitions. The ratio K of planning steps to real steps is the lever that controls how heavily the agent exploits its learned model between actions, and is the first explicit statement of the sample-efficiency claim that modern model-based work continues to revisit; the MBPO short-rollout analysis in Janner et al. (2019) is in the same family. We defer the algorithm block to §0.3, where we develop Dyna-Q in the notation of this survey.

The architectural openness of Dyna is as important as its sample-efficiency claim. Sutton states explicitly that the learned model can be probabilistic and oftentimes incorrect, with the planner doing useful work as long as the cost of synthetic experience is small relative to the cost of real experience. No structure is imposed on the class of admissible

models, on the loss used to fit them, or on the planner that consumes their output. This is the architectural slot that subsequent work fills with tabular counts, Gaussian processes, deep ensembles, recurrent state-space models, and implicit latent dynamics, each plugged into the same outer loop. The conceptual shift from a fixed control algorithm to an open architecture with a model component is what makes Dyna the entry point for the line that runs through §0.6 and §0.8.

0.2.2 Schmidhuber’s controller-model architecture

Schmidhuber (1990) proposes a paired architecture consisting of a recurrent model network M and a controller network C . The model M receives the controller’s outputs together with current observations and is trained, self-supervised, to predict the next inputs to C , including sensory observations and reinforcement signals. The controller C produces actions and is trained jointly with M . Once M is a reasonable forecaster, the environment becomes differentiable from the controller’s perspective, since the Jacobian of M stands in for the unknown Jacobian of the true dynamics. Planning is then implemented as gradient descent through the model network, by unrolling C and M together for a fixed horizon and descending on a predicted pain or cost signal, without executing any real actions during the planning phase. This is the first proposal of analytic gradient backpropagation through a learned dynamics model for policy improvement.

In the notation of this survey, let M_θ parameterize the world model with $p_\theta(s_{t+1} \mid s_t, a_t)$ and let C_ϕ parameterize the controller with $\pi_\phi(a_t \mid s_t)$. The model is fit on observed transitions by the negative log-likelihood

$$\mathcal{L}_M(\theta) = - \sum_t \log p_\theta(s_{t+1} \mid s_t, a_t),$$

and the controller is fit by maximizing the expected discounted return under the *learned* model,

$$J(\phi) = \mathbb{E}_{a_t \sim \pi_\phi(\cdot \mid s_t), s_{t+1} \sim p_\theta(\cdot \mid s_t, a_t)} \left[\sum_t \gamma^t r_t \right],$$

with the planning operator $\phi \leftarrow \phi + \eta \nabla_\phi J(\phi)$ implemented by backpropagating through the H -step unrolled M_θ via reparameterized or score-function gradients. The curiosity unit adds an intrinsic reward $r_t^{\text{int}} = \|s_{t+1} - \mathbb{E}_{M_\theta}[s_{t+1} \mid s_t, a_t]\|^2$ to the extrinsic reward inside J , with M_θ trained to drive precisely this prediction error toward zero so that

regions of state space where M_θ is currently uncertain attract C_ϕ . The idea reappears in the deep era as the intrinsic curiosity of Pathak et al. (2017), instantiated with a forward-dynamics network in a learned feature space and used to drive exploration in sparse-reward video games; the full deep realization of the controller-model architecture is the work of Ha and Schmidhuber (2018) and is treated in §0.4.

The contrast with Sutton’s Dyna is sharp on two dimensions and shallow on a third. The model class differs, a deterministic tabular Dirac in Dyna versus a differentiable neural network in Schmidhuber. The planner differs, K tabular Q -updates per real step in Dyna versus a gradient step on ϕ through the unrolled M_θ in Schmidhuber. The outer loop is the same: learn M from real data, use M to improve the policy, act in the world.

The symmetry between the two 1990 papers is what makes them an instructive joint origin. Both describe an agent that learns a forecaster of its environment and uses simulated experience from that forecaster to improve its policy, and both do so without the empirical tools that would let the idea work at the scale of pixel inputs or continuous control benchmarks. Both anticipate components that took roughly three decades to mature into reliable practice, intrinsic curiosity in one direction and planning amplification through short model rollouts in the other. In the next subsection we develop Dyna-Q in the notation of this survey and state the empirical claim that the planning-step parameter K trades real samples for synthetic ones, which is the cleanest formal target the 1990 papers leave for the chapters that follow.

0.3 Dyna-Q in Modern Notation

The previous section traced two parallel origins of model-based reinforcement learning in 1990, one from the dynamic-programming tradition and one from the recurrent-neural-network tradition. This section develops the canonical instantiation of Sutton’s side of that origin story, the Dyna-Q algorithm, in proper notation. Dyna-Q combines tabular Q -learning with a one-step deterministic model of the environment and applies K planning updates per real interaction step. It is the template that the deep-learning era inherits and modifies, and it is the reference point that the algorithms chapter (§??) points to whenever model-based methods enter the discussion. The section gives the algorithm in formal notation, states the empirical headline from Sutton’s original gridworld experiment, and then isolates the two architectural commitments that survive every subsequent

reformulation of the idea.

0.3.1 Algorithm

Fix a finite Markov decision process with state space $\mathcal{S}$ and action space $\mathcal{A}$. Let $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ denote the tabular action-value function the agent maintains over the course of interaction. Let $\hat{P}(s' | s, a)$ denote the learned transition model and $\hat{r}(s, a)$ the learned reward model; together these form the agent's internal forecaster of its environment. The remaining objects are a step size $\alpha \in (0, 1]$, a discount factor $\gamma \in [0, 1)$, an exploration parameter $\varepsilon \in (0, 1)$, and a planning horizon $K \in \mathbb{N}$ counting the number of synthetic updates applied per real step. The agent initializes Q , $\hat{P}$, and $\hat{r}$ to arbitrary values and then proceeds as follows.

For each step t :

1. Observe the current state s and select an action $a \in \arg \max_{a'} Q(s, a')$ with ε -greedy exploration, breaking ties uniformly at random.
2. Execute a in the environment and observe the reward r and the next state s' .
3. Direct RL update,

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a''} Q(s', a'') - Q(s, a) \right].$$

4. Model update, $\hat{r}(s, a) \leftarrow r$ and $\hat{P}(\cdot | s, a) \leftarrow \delta_{s'}$, recording the most recent observed transition as a deterministic prediction.
5. Planning. Repeat K times, on each iteration sampling a previously-visited pair $(\tilde{s}, \tilde{a})$ uniformly from the agent's experience, querying $\hat{r}(\tilde{s}, \tilde{a})$ and $\hat{P}(\cdot | \tilde{s}, \tilde{a})$ to obtain a synthetic transition $(\tilde{r}, \tilde{s}')$, and applying the same Q -update as in step 3,

$$Q(\tilde{s}, \tilde{a}) \leftarrow Q(\tilde{s}, \tilde{a}) + \alpha \left[\tilde{r} + \gamma \max_{a''} Q(\tilde{s}', a'') - Q(\tilde{s}, \tilde{a}) \right].$$

Step 3 is the tabular Q -learning update of [Watkins and Dayan \(1992\)](#), who prove that under decreasing step sizes satisfying $\sum_t \alpha_t = \infty$ and $\sum_t \alpha_t^2 < \infty$ and infinite visitation of every state-action pair, Q converges with probability one to the optimal action-value Q^* , the fixed point of the Bellman optimality operator $\mathcal{T}^*$ defined by $(\mathcal{T}^*Q)(s, a) = \mathbb{E}_{s' \sim P^*(\cdot | s, a)}[r(s, a) + \gamma \max_{a'} Q(s', a')]$. The operator is a γ -contraction in the sup norm, which is the technical ingredient that powers the proof. Step 5 applies the same update

with P^* and r replaced by the learned $\hat{P}$ and $\hat{r}$; convergence of the full planning-and-learning loop has no closed-form guarantee in the tabular case beyond the consistency of $\hat{P}$ and $\hat{r}$ under the agent’s visitation distribution, and the function-approximation setting is studied via Lipschitz-on-model arguments (Asadi et al. 2018b, discussed in §0.10.2) rather than by a single contraction.

The conceptual content of the indirect-versus-direct distinction collapses into a single observation. Steps 3 and 5 apply identical update rules to identical-looking (s, a, r, s') tuples. The only difference is the provenance of the tuple. In step 3 the tuple is sampled from the true transition kernel of the environment by acting; in step 5 the tuple is sampled from the agent’s learned model. Planning, in this architecture, is not a separate algorithm with its own update rule. It is the same update rule applied to model-generated rather than environment-generated experience, and the ratio K is the single lever that controls how heavily the agent leans on its model relative to fresh interaction.

0.3.2 Empirical headline

Sutton (1990) reports a gridworld maze experiment that fixes the empirical claim. The agent navigates a tabular maze with deterministic transitions and sparse reward, learning under three settings of the planning horizon, $K \in \{0, 5, 50\}$. Setting $K = 0$ recovers plain Q -learning, since the planning loop is skipped and only the direct RL update of step 3 fires per interaction. With $K = 50$, the agent reaches near-optimal performance in roughly three episodes; with $K = 0$, the same level of performance requires roughly twenty-five episodes. The ratio of episodes is the original planning-amplification claim. Each real step in the $K = 50$ variant carries the policy-improvement value of about fifty steps in the $K = 0$ variant, at the cost of additional internal computation and a learned model that must be accurate enough for the synthetic updates to be informative. A blocking-maze variant in the same paper shows that a small exploration bonus permits the agent to recover after the environment changes, addressing the obvious failure mode that comes with relying on a learned model.

0.3.3 Two architectural commitments

The first commitment is that the model class is left unspecified. The algorithm box above uses a deterministic one-step Dirac model $\hat{P}(\cdot \mid s, a) \leftarrow \delta_{s'}$, but this choice is incidental to

the architecture. Sutton describes the model as “probabilistic and oftentimes incorrect,” which is the conceptual move that allows every subsequent paper to plug a different model class into the same outer loop without changing anything else. The deep era exercises this openness across diverse model classes. [Ha and Schmidhuber \(2018\)](#) use a convolutional variational autoencoder paired with a mixture-density recurrent network, treated in §0.4. The Recurrent State-Space Model that powers the PlaNet and Dreamer lines, treated in §0.5, couples a deterministic recurrent path with a stochastic latent. MuZero, the value-aware end of the spectrum treated in §0.7, dispenses with observation reconstruction entirely and trains the model only against quantities that enter the planner’s value calculation. The MBPO line, treated in §0.6, uses a deep ensemble of probabilistic feed-forward networks. The outer loop in each of these is recognizably Dyna; the model class is the variable that distinguishes them.

The second commitment is that the inner loop applies the same Bellman update on simulated and real data. Step 5 of the algorithm is not a distinct planning operator; it is step 3 with a different source of transitions. This single fact is what permits Dyna to unify direct and indirect reinforcement learning into one architecture rather than two competing approaches. In the dynamic-programming tradition before Sutton, indirect methods estimated a model and then ran value iteration on it as a separate batch computation, while direct methods updated values from interaction without any model. Dyna eliminates the wall between these two camps by routing both kinds of transitions through the same temporal-difference update. The architectural consequence is that the agent does not need to choose between learning from interaction and learning from a model; it can do both, and the ratio K tunes the mixture. Every subsequent model-based method in this chapter inherits this commitment, whether it uses tabular updates, fitted Q -iteration, soft actor-critic, or an MPC controller as its inner loop. Read in economics terms, Dyna is adaptive learning with simulated experience added between observations. The sample-efficiency claim that follows is not a free lunch; simulated experience is useful only where the learned model is accurate enough for the planner’s purpose, and most of the failure modes the chapter encounters later are forms of that purpose-relative inaccuracy.

0.3.4 Simulation Study: Planning Amplification in the Blocking Maze

This study is the chapter’s first empirical study and is designed to make three claims quantitative. The first is the planning-amplification claim of Sutton (1990): that on the same maze and the same environment-step budget, the value of each real interaction grows roughly in proportion to the number of model-based planning updates applied between interactions. The second is the diagnosis of why plain ε -greedy Q -learning is too slow on a sparse-reward tabular problem of this size and discount, an observation the literature often takes as given without quantification. The third is the model-staleness failure mode that comes with relying on a learned model; the experiment flips the wall mid-experiment to expose whether agents that trust their old model can recover. Figure 1 renders the two phases of the environment side by side.

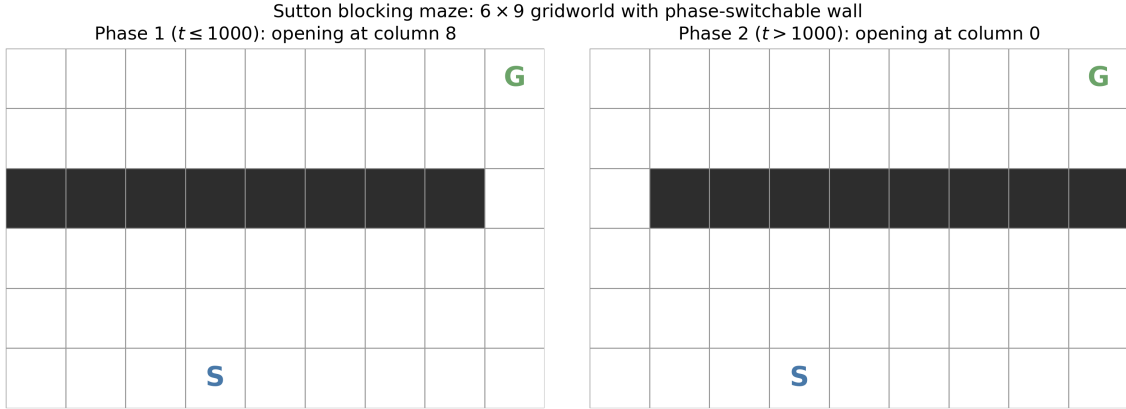


Figure 1: Sutton blocking maze. The agent starts at S and aims for the goal at G ; the only opening in the wall along row two shifts from column eight in Phase 1 to column zero in Phase 2 at environment step $t = 1000$. The flip is not signaled to the agent and triggers the model-staleness failure mode that the experiment is designed to expose.

Why plain Q -learning is slow on this maze. The diagnosis is quantitative. With $\gamma = 0.95$, $\alpha = 0.1$, $\varepsilon = 0.1$, Q -values initialized to zero, and a reward signal that is identically zero everywhere except at the goal, the agent reaches its first goal essentially by random walk; the expected hitting time of a single random walker from S to G on the open part of this 6×9 grid is several hundred steps. Each goal-reach updates exactly one entry of Q by $\alpha \cdot 1 = 0.1$ toward the optimal value γ^{d-1} , where d is the path length from the parent state. Propagating that signal back to the start state requires a chain of subsequent goal-reaches at the right places, and ε -greedy noise at $\varepsilon = 0.1$ diverts the

agent off its currently-best path on one in ten steps. The chance of stringing together a long enough sequence of goal-reaches to back the value all the way to S within a few hundred environment steps is correspondingly small. This is the precise form of the slow-per-step-propagation claim that Dyna-Q is designed to address by replaying each successful real transition K times against the model between every pair of real steps.

Setup. The environment is the 6×9 deterministic gridworld of Sutton and Barto (2018), with start at row five column three and goal at row zero column eight. Actions are the four cardinal moves; transitions are deterministic; the reward is one on goal arrival and zero elsewhere; the discount factor is $\gamma = 0.95$. The wall along row two has its only opening at column eight during Phase 1 ($t \leq N_1$ with $N_1 = 1000$) and at column zero during Phase 2 ($N_1 < t \leq N_1 + N_2$ with $N_2 = 2000$). The phase flip happens automatically at $t = N_1$ and is not signaled. The total interaction budget is $N_1 + N_2 = 3000$ environment steps, the per-episode step cap is two hundred, and the experiment runs over thirty independent seeds with $\alpha = 0.1$, $\varepsilon = 0.1$, and zero-initialized Q .

Models. Five agents share the budget. Plain Q -learning with $K = 0$ is the direct-RL baseline, applying only the temporal-difference update of Watkins and Dayan (1992) per real step. Dyna-Q with $K = 5$ applies five model-based replays per real step from a tabular Dirac model, and Dyna-Q with $K = 50$ applies fifty; both implement the algorithm box of §0.3.1 with the planning-step count set to the named value. Dyna-Q+ with $K = 50$ adds the exploration bonus $\kappa\sqrt{\tau(s,a)}$ to planning targets, with $\tau(s,a)$ the time since the state-action was last visited and $\kappa = 10^{-4}$, instantiating the primitive curiosity mechanism that connects to the prediction-error and ensemble-disagreement variants of §0.10.2. The fifth agent applies REINFORCE on imagined rollouts under a learned forward model, related to the Schmidhuber 1990 controller-model architecture of §0.2.2 but distinct in the gradient pathway, since the original 1990 formulation propagates analytic gradients through a differentiable model into the controller while the variant used here treats the rollout as a stochastic-policy estimator. The agent uses two small multilayer perceptrons: a model network M_θ mapping one-hot state plus one-hot action to next-state logits, fit on observed transitions by cross-entropy, and a controller network C_ϕ mapping one-hot state to action logits, fit by a REINFORCE policy-gradient estimator on imagined trajectories rolled forward through M_θ . Both networks have a single thirty-

two-unit hidden layer; planning calls happen every ten real steps with sixteen imagined trajectories of length ten and a moving-average baseline. This is the third 1990-era treatment of the same loop, sitting alongside Sutton’s tabular Dyna and the two Dyna-Q variants of $K \in \{5, 50\}$.

Results. Figure 2 and Table 1 report cumulative environment reward across the thirty seeds with mean and standard error bands, with rows in rank order by total reward at $t = 3000$. Dyna-Q at $K = 50$ leads with 52.0 ± 4.2 total goal-reaches, Dyna-Q+ at $K = 50$ follows at 47.0 ± 4.4 , Dyna-Q at $K = 5$ at 39.2 ± 4.7 , the Schmidhuber controller-model agent at 4.0 ± 0.4 , and plain Q -learning closes the field at 3.5 ± 0.5 . The top of the table reproduces the planning-amplification claim of Sutton (1990) cleanly under this budget; each real interaction in the $K = 50$ run carries roughly an order of magnitude more policy-improvement weight than the same interaction in $K = 0$, because the synthetic updates of Step 5 propagate the learned reward back through the state graph between every pair of real steps. Dyna-Q+ at $K = 50$ pays a small Phase 1 cost of about five reward units relative to plain Dyna-Q because the exploration bonus eats slightly into exploitation. The cost remains modest because the implementation follows Sutton and Barto (2018) §8.3 and registers all four actions at every visited state with $\tau = 0$ on first encounter, so the bonus drives discovery of untried actions rather than only perturbing the values of already-tried ones. The Schmidhuber agent tracks plain Q -learning on this maze rather than tabular Dyna-Q because the gradient-based fit of the neural model needs many real samples to specialize from a random initialization, and at three thousand environment steps it has not yet accumulated enough buffer entries near the goal for the REINFORCE estimator to drive the controller toward the discovered path. Phase 2 reveals the recovery story. Dyna-Q at $K = 50$ continues to gain reward at the rate of 9.8 ± 4.0 over the remaining two thousand steps because ε -greedy noise eventually exposes the new corridor and the planning loop quickly retrain the model. Dyna-Q+ matches that rate at 9.6 ± 2.8 , with the curiosity bonus driving the agent to revisit untried actions on the opposite side of the wall. The two rates are statistically indistinguishable on a Welch t -test with $p > 0.5$, and the Phase 1 deficit shrinks but does not close by $t = 3000$. Sutton and Barto (2018) Figure 8.5 reports Dyna-Q+ recovering faster than Dyna-Q after the maze flip; the thirty-seed reproduction here does

not separate the two methods on the recovery slope, and a longer post-flip budget or a larger bonus coefficient than $\kappa = 10^{-4}$ would likely be needed to recover the Fig. 8.5 effect size cleanly. The Schmidhuber agent’s Phase 2 gain of 1.5 ± 0.2 is only modestly above plain Q -learning’s 0.6 ± 0.2 and tracks the same slow trajectory. The verdict is that the planning-amplification claim of Sutton (1990) reproduces cleanly under this budget, with tabular Dyna-Q at $K = 50$ delivering an order-of-magnitude improvement over $K = 0$, and Dyna-Q+ with faithful untried-action initialization recovering at the same Phase 2 rate as plain Dyna-Q while paying only a small Phase 1 exploration cost. The neural controller-model agent pays the cost of fitting a learned model from scratch on a tabular task that does not need a neural model to represent. The three 1990-era commitments, the deterministic Dirac model with K planning steps, the exploration bonus as a stale-model fix, and a learned neural model with rollout-based policy gradients, sit at three different points on the empirical frontier, with tabular Dyna-Q dominating at this scale and the neural-model architecture’s promise visible only when the state space is large enough that tabular methods cannot follow.

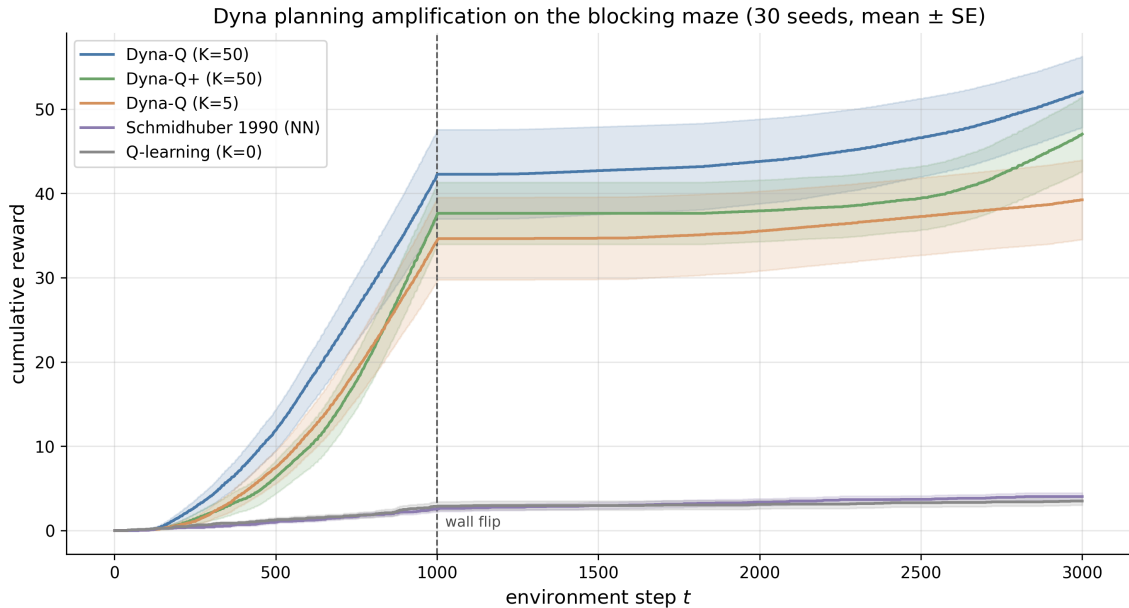


Figure 2: Cumulative reward on the blocking maze for five agents under a shared three-thousand-step environment budget. The dashed vertical line marks the wall flip at $t = 1000$. Lines are means and shaded bands are one standard error across thirty seeds.

Table 1: Cumulative reward on the blocking maze. End of Phase 1 is at $t = 1000$; Phase 2 gain is the increase over the remaining two thousand steps; Total is the value at $t = 3000$. Mean $\pm$ standard error across thirty seeds.

Agent	End of Phase 1	Phase 2 gain	Total ($t = 3000$)
Dyna-Q (K=50)	42.2 ± 5.3	9.8 ± 4.0	52.0 ± 4.2
Dyna-Q+ (K=50)	37.4 ± 3.7	9.6 ± 2.8	47.0 ± 4.4
Dyna-Q (K=5)	34.4 ± 4.9	4.8 ± 3.0	39.2 ± 4.7
Schmidhuber 1990 (NN)	2.5 ± 0.3	1.5 ± 0.2	4.0 ± 0.4
Q-learning (K=0)	2.9 ± 0.5	0.6 ± 0.2	3.5 ± 0.5

0.4 World Models: The Deep Revival

Ha and Schmidhuber (2018) is the first paper to make latent-space dreaming work at scale on pixel inputs. An agent learns a visual encoder, a recurrent forecaster of the encoder’s output, and a controller, with the controller trained against rollouts of the forecaster rather than against real environment frames. The abstract decomposition is a triple of objects with distinct losses, a vision encoder trained against reconstruction, a memory model trained against next-latent likelihood, and a controller trained against imagined returns; this decomposition is the architectural pattern that subsequent deep model-based methods inherit and modify, with the Dreamer line collapsing vision and memory into one jointly trained object and the value-aware line of §0.7 replacing the memory loss outright. On the CarRacing benchmark the resulting controller is the first to cross the conventional solution threshold; on a VizDoom task a controller trained entirely inside the learned forecaster transfers zero-shot to the real environment. The paper is an explicit homage to Schmidhuber (1990), the controller-model architecture introduced in §0.2.2, and the line of work it revives is twenty-eight years old. What is new is the toolkit. Convolutional variational autoencoders, mixture-density recurrent networks, and evolution strategies, taken together, make the older architecture empirically tractable at a scale the original could not reach.

Nine recurring terms anchor the rest of the chapter. A *world model* is an estimated model of next states and rewards. A *planning update* is a policy or value update performed using the learned model rather than the environment. A *simulated rollout* is a trajectory generated by the learned model rather than by real interaction. A *latent state* is an internal compressed state used by a neural model, not an observed economic state. A *model loss* is the criterion used to train the world model. A *value-aware loss* is a model

loss that cares about preserving values rather than predicting every detail of the next observation. A *short rollout* is a deliberately short simulated trajectory from a real visited state. A *bootstrap* is a value estimate that stands in for rewards beyond the simulated horizon. A *model error* is the gap between the learned model and the true environment, judged by its effect on planning rather than by its statistical fit.

0.4.1 Three components

The vision component V is a convolutional variational autoencoder.¹ The encoder maps an observation $x_t \in \mathbb{R}^{64 \times 64 \times 3}$ to a latent vector $z_t \sim \mathcal{N}(\mu(x_t), \sigma(x_t)^2 I)$ with $z_t \in \mathbb{R}^{N_z}$, where $N_z = 32$ for CarRacing and $N_z = 64$ for VizDoom. The decoder is trained jointly with the encoder against the standard ELBO objective,² combining a per-pixel reconstruction term with a Kullback-Leibler penalty on the latent. The vision component is trained on observation reconstruction alone, independently of the controller, on a buffer of frames collected by a random policy. The decoder is discarded once training is complete; downstream components see only the encoder’s latents. The architectural commitment is that all subsequent processing happens in a low-dimensional vector space rather than in pixel space, which is what makes the rest of the pipeline cheap enough to train and to roll out.

The memory component M is a mixture-density recurrent neural network.³ Its body is an LSTM with hidden state h_t ;⁴ its head outputs the parameters of a Gaussian mixture over the next latent, conditional on the current action, the current latent, and the recurrent state, modeling $P(z_{t+1} \mid a_t, z_t, h_t)$ as a mixture of five diagonal Gaussians. A temperature parameter on the mixture controls how aggressively the forecaster samples from its tails, and is exposed as a tuning lever for controller training. The memory component is trained on next-latent likelihood alone, again independently of the controller, using the same buffer of rollouts under a random policy that trained V . The mixture form matters in environments with discrete event structure such as enemy fireballs in

¹A variational autoencoder compresses high-dimensional observations into a low-dimensional latent vector by training an encoder and a decoder jointly.

²Evidence lower bound; the standard variational autoencoder training objective that combines a reconstruction term with a Kullback-Leibler penalty between the encoder’s latent and a fixed prior. The Kullback-Leibler term measures the discrepancy between two distributions.

³A mixture-density recurrent network is a recurrent network whose output parameterizes a Gaussian mixture over the next latent rather than a single mean; this matters when the next state is genuinely multimodal.

⁴Long short-term memory; a gated recurrent cell that propagates information across time and is the dominant pre-Transformer architecture for sequential data.

VizDoom, where the next latent has multiple plausible values that a unimodal Gaussian forecaster would average over and so render unusable for planning.

The controller C is a single linear layer. It outputs an action $a_t = W_c[z_t, h_t] + b_c$ from the concatenation of the visual latent and the recurrent state. On CarRacing the controller has roughly 870 parameters, compared to about 5 million in V and M together. It is trained by covariance matrix adaptation evolution strategy,⁵ evaluating candidate parameter vectors against rollouts drawn from M rather than against real environment frames. Evolution strategies sidestep the need for differentiability through M at the cost of higher sample complexity per parameter, which the linear policy keeps manageable. The controller is the only component whose loss depends on reward, and it is the only component whose training requires interaction, in the form of imagined trajectories through the memory network.

0.4.2 Empirical headline

On CarRacing-v0 the full agent scores 906 ± 21 over 100 random tracks. The benchmark requires an average above 900 to count as solved, and this is the first reported solution. On the VizDoom DoomTakeCover task the controller is trained inside the memory network, with the visual encoder and recurrent forecaster fitted on a buffer of random rollouts and the controller never exposed to real frames during policy search. Transferred zero-shot to the real environment, the controller scores 1092 ± 556 against a solution threshold of 750. The variance is large but the mean clears the threshold, and the policy is constructed without any direct contact with the real environment after the data collection phase.

0.4.3 Architectural reading

The modular training schedule, with V fitted on reconstruction, M fitted on next-latent likelihood, and C fitted on dream rollouts, is the practical workaround for joint end-to-end training, which the paper notes is hard at this scale. Each component has its own loss, its own optimizer, and its own data pipeline, and the three are composed in sequence rather than co-trained. The cost of the workaround is that the representation learned by V is agnostic to what the controller will use it for, and the forecaster learned

⁵A gradient-free black-box optimizer that maintains a Gaussian over candidate parameter vectors and updates the Gaussian by re-fitting to the best-scoring samples each generation.

by M is agnostic to what the controller will reward. The line of work that eventually unifies V and M into a single jointly-trained recurrent state-space model is the PlaNet and Dreamer family, treated in §0.5.

The combination of V and M does the representation work; the controller is tiny by comparison and carries the parameter budget of a linear policy. This anticipates the modern observation that a strong world model lets a small actor produce competitive policies, a point pressed further by Dreamer in §0.5 and by TD-MPC2 in §0.8, where the latent space and the value function carry most of the inductive load and the policy network is relatively light. The training-in-dream-then-transfer protocol on VizDoom is the strongest demonstration to date that a learned forecaster can stand in for the environment during policy search, with the agent’s only contact with the real environment being the random data collection used to fit V and M in the first place.

0.5 The Recurrent State-Space Model and the Dreamer Line

The previous section described the modular Vision-Memory-Controller architecture of Ha and Schmidhuber (2018), trained in three separate stages. The Recurrent State-Space Model collapses that sequence into a single jointly trained network and becomes the workhorse architecture of modern world-model reinforcement learning. The line carries through PlaNet, DreamerV1, DreamerV2, and DreamerV3 and is treated here in version-agnostic notation; the differences across versions are summarized at the end of the subsection. The pivot at the end is to the question of what loss the model should be trained against, which the next section picks up.

0.5.1 The Recurrent State-Space Model

Let o_t denote the observation at time t , a_t the action, and r_t the reward. The Recurrent State-Space Model factors the latent state into a deterministic recurrent component h_t and a stochastic latent s_t , coupled by

$$h_t = f_\theta(h_{t-1}, s_{t-1}, a_{t-1}), \quad s_t \sim p_\theta(s_t \mid h_t).$$

The deterministic path f_θ is a recurrent cell (in practice a Gated Recurrent Unit); the prior $p_\theta(s_t \mid h_t)$ is either a diagonal Gaussian or a product of categoricals, the choice a

design parameter rather than a structural commitment. A posterior network $q_\theta(s_t \mid h_t, o_t)$ encodes the current observation into a refined latent for inference at training time. The joint loss for a length- T trajectory has three named terms,

$$\mathcal{L}_\theta = \underbrace{-\sum_t \log p_\theta(o_t \mid h_t, s_t)}_{\text{reconstruction}} - \underbrace{\sum_t \log p_\theta(r_t \mid h_t, s_t)}_{\text{reward log-likelihood}} + \underbrace{\sum_t \text{KL}[q_\theta(s_t \mid h_t, o_t) \parallel p_\theta(s_t \mid h_t)]}_{\text{prior-posterior consistency}}.$$

Optionally a latent-overshooting term enforces multi-step consistency in latent space without decoding. The point of the architecture is to separate deterministic memory from stochastic uncertainty so that the model forecasts a distribution over futures while still propagating an information bottleneck through time. The model class is a flexible neural object, but the algorithmic outer loop is recognizably the Dyna template of §0.3, with $\hat{P}$ and $\hat{r}$ replaced by jointly trained networks.

0.5.2 Actor-critic in imagination

The policy is trained inside the learned world model rather than from environment interaction directly. An actor $\pi_\phi(a_t \mid h_t, s_t)$ and a critic $V_\psi(h_t, s_t)$ are updated on imagined trajectories rolled forward through the latent dynamics, starting from real states drawn from the agent’s replay buffer. The imagined return objective is

$$J(\phi) = \mathbb{E}_{\pi_\phi, p_\theta} \left[\sum_{\tau=t}^{t+H} \gamma^{\tau-t} \hat{r}_\tau + \gamma^{H+1} V_\psi(h_{t+H+1}, s_{t+H+1}) \right],$$

where the expectation is taken over actions sampled from the actor and latent states evolved by the world model, H is an imagination horizon, $\hat{r}_\tau$ is the reward predicted by the trained reward head, and the bootstrap term V_ψ supplies a value estimate past the imagination horizon. Because the world-model dynamics are differentiable and the latent samples are reparameterized, $\nabla_\phi J$ is obtained by backpropagating value gradients through the unrolled latent trajectory. The critic is fit by minimizing a squared error against λ -return targets in the same imagined trajectories.

This is the deep-learning instantiation of the inner planning loop of Dyna-Q, with the model now a flexible neural object and the planner an analytic policy-gradient operator rather than a discrete Q-update. Convergence guarantees are not available in closed form for this setting; the nearest analogue is the Lipschitz contraction bound for model-based

value error of Asadi et al. (2018b), discussed in §0.10.2, which states that value error grows geometrically along the rollout at rate K_F (the Lipschitz constant of the learned dynamics) and remains finite when $\gamma K_F < 1$.⁶

The line trains the world model against likelihood-style losses (reconstruction, reward log-likelihood, prior-posterior consistency), which rewards the model for matching observed distributions irrespective of how those distributions enter the value calculation. Read in economics terms, the recurrent state-space model is a high-dimensional perceived law of motion, fit by likelihood rather than by hand-chosen moments, with a planner that operates entirely on the learned latent. The next section asks whether this is the right objective. The argument, developed by Farahmand et al. (2017) and Grimm et al. (2020) and instantiated empirically in MuZero, is that the model should be trained against the value functional directly rather than against pixel and reward likelihoods. The Dreamer line achieves its results despite training to the wrong target; the value-aware line asks how much further the field could go if the loss were aligned with the downstream decision.

0.6 Short Rollouts and Ensembles: The Modern Dyna

The line of work culminating in Janner et al. (2019) is the continuous-control inheritor of Dyna (§0.3). The architectural recipe is a deep ensemble of probabilistic dynamics models paired with a model-free off-policy actor-critic, soft actor-critic in the canonical implementation, with synthetic experience generated by branching short imagined rollouts from real states drawn out of the replay buffer. The framing slogan, present in the paper and in the surrounding literature, is to not trust the model too far. Breaking the dependence between the task horizon and the model rollout horizon is the move that makes the model-error penalty tractable, and is what separates this line from the trajectory-optimization approach of Chua et al. (2018) and the on-policy approach of Kurutach et al. (2018). The conceptual lineage runs back to Sutton (1990), with the same outer loop of direct learning from real transitions and indirect learning from imagined ones, only with a deep ensemble in place of the tabular model and an off-policy

⁶The four canonical instantiations of the architecture cover the major design choices. PlaNet (Hafner et al., 2019) introduces the RSSM with cross-entropy planning. DreamerV1 (Hafner et al., 2020) replaces planning with actor-critic in imagination. DreamerV2 (Hafner et al., 2021) switches the Gaussian latent for categoricals with a Kullback-Leibler balancing trick, surpassing top model-free agents at the Atari 200M budget. DreamerV3 (Hafner et al., 2025) fixes one hyperparameter configuration across more than one hundred and fifty tasks via symlog reward normalization and free-bits Kullback-Leibler, including the Minecraft diamond from scratch.

actor-critic in place of tabular Q-learning.

0.6.1 Branched short rollouts

As an abstract operator, the branched rollout maps the pair (real-state distribution, learned dynamics) to a synthetic transition distribution of length k , and decouples the task horizon from the model horizon by anchoring every imagined trajectory at a state the agent has actually visited. The MBPO branching scheme is straightforward. At each gradient step the algorithm samples a real state s from the replay buffer, rolls the learned dynamics ensemble forward for k steps from s under the current policy, and trains the actor-critic on the resulting model-generated transitions alongside real ones. The actor-critic update does not distinguish synthetic transitions from real transitions; the only architectural change relative to model-free soft actor-critic is the distribution from which training tuples are drawn. Real states act as anchors. Because every imagined trajectory is no more than k steps long and originates at a state the agent has actually visited, the model is never asked to extrapolate from its own predictions for more than k steps, even when the task itself runs for hundreds of steps.

The returns bound in Janner et al. (2019, Theorem 4.2) formalizes the trade-off. With ϵ_m the model error measured in total variation, ϵ_π the policy distribution shift relative to the data-collection policy, k the branching length, and $r_{\max}$ the reward bound,

$$\eta[\pi] \geq \eta^{\text{branch}}[\pi] - 2r_{\max} \left[\frac{\gamma^{k+1}\epsilon_\pi}{(1-\gamma)^2} + \frac{\gamma^k + 2}{1-\gamma}\epsilon_\pi + \frac{k}{1-\gamma}(\epsilon_m + 2\epsilon_\pi) \right].$$

The first two terms penalize policy shift and shrink in k ; the third penalizes model error and grows linearly in k . Theorem 4.3 then shows that an optimal $k^\star > 0$ exists whenever the model error is small enough, justifying short rollouts as the right way to spend a fixed model-error budget. The practical takeaway, stated in the paper’s ablations, is that $k = 1$ is competitive across the MuJoCo suite, that k in the range of roughly five to fifteen typically gives the best returns, and that values much beyond about 25 are too inaccurate to help. The empirical sweet spot is well inside the regime where the model is treated as a local data augmentation rather than as a long-horizon simulator.

Read in economics terms, the branched-rollout architecture is the computational version of using a locally valid structural model for counterfactual policy search; the model

is trusted only over short windows around states the agent has already visited, and the rest of the long-horizon credit is left to the off-policy critic. The dynamics ensemble used in this line, typically five to seven neural networks each predicting a Gaussian over the next-state delta, plays two roles. As an aggregate predictor the ensemble averages over members and trims tails to give a calibrated forecast that a single deterministic network would not produce; as a disagreement signal the spread across members is a cheap proxy for epistemic uncertainty, large where the agent has not visited often and small where coverage is good.⁷⁸

0.7 What Should the Model Be Trained Against?

Throughout the preceding subsections the model has been trained against a likelihood or reconstruction loss. The Recurrent State-Space Model line uses a pixel reconstruction term, a reward log-likelihood, and a Kullback-Leibler consistency term between prior and posterior; the original Dyna model is a tabular maximum-likelihood estimate of the empirical transition; the controller-model architecture of [Ha and Schmidhuber \(2018\)](#) fits its variational autoencoder and its mixture-density recurrent network to observation and next-latent likelihoods respectively. In each case the loss rewards the model for matching observed distributions, regardless of how those distributions enter the value calculation the agent actually optimizes. The objection raised in the present subsection is that the agent does not care about predicting the next observation perfectly; it cares about getting the value function right. This subsection traces the line that took that observation seriously, from value-aware model learning in [Farahmand et al. \(2017\)](#) through the value-equivalence principle of [Grimm et al. \(2020, 2021\)](#), the value-targeted regret bound of

⁷⁸The ensemble appears in two prior methods with different commitments. [Kurutach et al. \(2018\)](#) pairs the ensemble with on-policy trust-region policy optimization, estimating value gradients while clipping updates that would push the data distribution into regions where members disagree; the on-policy constraint bounds ϵ_π explicitly but keeps the long-horizon dependence on the model. [Chua et al. \(2018\)](#) wraps the same ensemble in model-predictive control and bypasses policy learning entirely by propagating particle trajectories through ensemble members at planning time, which solves the horizon problem by conflating the task and planning horizons and limits scaling to long-horizon tasks where MPC becomes prohibitive. MBPO’s branched-rollout architecture pushes the two terms of the returns bound into a regime where standard off-policy machinery can absorb them, and the same recipe carries from low-dimensional locomotion to humanoid without changes to the outer loop.

⁸On the MuJoCo continuous-control benchmark [Janner et al. \(2019\)](#) matches the asymptotic return of soft actor-critic with roughly an order of magnitude fewer environment samples; the Ant task reaches the soft actor-critic return at around 300,000 real steps against the latter’s roughly 3,000,000, and on Humanoid (a 376-dimensional state with a 17-dimensional action) MBPO converges where PETS fails to learn, consistent with the architectural reading that the task horizon dwarfs PETS’s planning horizon. [Kurutach et al. \(2018\)](#) reports a similar order-of-magnitude sample-efficiency gain for ME-TRPO over TRPO/PPO/DDPG on the same suite.

Ayoub et al. (2020), the MuZero instantiation of Schrittwieser et al. (2020), and the recent calibration analysis of Voelcker et al. (2025).

0.7.1 Value-aware model learning

Farahmand et al. (2017) formalize the objection as a loss. Fix a value function class $\mathcal{V}$, a data distribution μ over state-action pairs, and a model class within which the learner selects an estimate $\hat{P}$ of the true transition kernel P^* . The value-aware model learning loss is

$$\mathcal{L}_{\text{VAML}}(\hat{P}) = \sup_{V \in \mathcal{V}} \left\| (\hat{P}V)(\cdot) - (P^*V)(\cdot) \right\|_{\mu},$$

where $(\hat{P}V)(s, a) = \int V(s') \hat{P}(ds' | s, a)$ is the expected next-state value under the learned model and $(P^*V)(s, a)$ is the same quantity under the true kernel. The loss penalizes the worst-case discrepancy in expected next-state value across $\mathcal{V}$, measured under μ . A model that places mass on features of the next observation irrelevant to every value function in $\mathcal{V}$ is not penalized; a model that distorts the value expectation is. Farahmand (2018) embed the same idea inside approximate value iteration, replacing the supremum over a value class with the realized value iterate at each step, which yields the practical IterVAML estimator and a fitted error bound that propagates through the Bellman backups rather than through the model alone. Asadi et al. (2018a) read the construction through optimal transport, observing that under a Lipschitz assumption on the value class the VAML loss reduces to a Wasserstein distance between $\hat{P}$ and P^* , which connects the value-aware loss to the substantial empirical literature on Wasserstein generative modeling.

VAML is the formal statement of the intuition that the model should be wrong in ways that do not matter for the value functional. This is the technical answer to the conceptual openness of Sutton’s original Dyna architecture, in which the model was permitted to be probabilistic and oftentimes incorrect (§0.3.3) without specifying what kind of error was tolerable. The likelihood-trained Recurrent State-Space Model line (§0.5) commits to a particular notion of error, namely the reconstruction loss on pixels and the Kullback-Leibler divergence on latents, and violates the value-aware principle whenever the pixel distribution is rich in detail orthogonal to the reward. Whether this matters in practice depends on the task; the empirical claim of the value-aware line is that it does, and that the gap widens as observation dimensionality grows.

0.7.2 Value equivalence and MuZero

Grimm et al. (2020) sharpen the VAML idea into a clean equivalence relation. Fix a policy set Π and a value function set $\mathcal{V}$. Two transition kernels $\hat{P}$ and P^* are value-equivalent with respect to $(\Pi, \mathcal{V})$ when their Bellman operators agree on every pair $(\pi, V) \in \Pi \times \mathcal{V}$, that is when $(T_{\hat{P}}^{\pi}V)(s) = (T_{P^*}^{\pi}V)(s)$ for all s and all (π, V) in the chosen sets. The equivalence class shrinks monotonically as Π and $\mathcal{V}$ grow, collapsing to the singleton $\{P^*\}$ in the limit. The practical implication is that a small model class can be sufficient for a restricted set of policies and value functions, even if it would be insufficient as a generative model of the environment. Grimm et al. (2021) subsequently introduce proper value equivalence, which restricts the value class to fixed points of the chosen Bellman operators and gives the variant of value equivalence that aligns most directly with the MuZero training objective. Ayoub et al. (2020) push the value-aware idea into the regret-bound literature with value-targeted regression, which trains the model against value targets and matches the dimension-dependent rates of optimism-based reinforcement learning under standard assumptions.

The empirical instantiation is MuZero. Schrittwieser et al. (2020) train a latent recurrence jointly with a policy network and a value network against three losses applied at the next step of the recurrence, namely a reward prediction loss, a value prediction loss, and a policy prediction loss matched to the output of a Monte Carlo tree search rooted at the same latent. There is no reconstruction loss; the latent dynamics are never asked to decode observations. The transition kernel itself has no explicit target, since the only quantities that enter any loss are the reward, the value, and the policy at the rollout step. Empirically the construction matches AlphaZero on Go, chess, and shogi without access to the rules of those games, and reaches state-of-the-art performance on the Atari benchmark. Antonoglou et al. (2022) extend the construction to stochastic transitions by inserting a learned afterstate node, the game-tree analogue of a chance node, between the agent action and the next state. Read in economics terms, value-aware model learning treats the world model as a task-relevant misspecification, kept correct on the value

functional even when it is wrong on the full observation distribution.⁹¹⁰

0.8 Convergence Point: TD-MPC2

TD-MPC2 (Hansen et al., 2024) is the convergence point of three lines that the preceding sections traced separately. Model predictive control with learned dynamics, in the style of PETS and MBPO (§0.6), supplies the planning frame. Recurrent latent world models, in the Dreamer line (§0.5), supply the idea that the model should live in a learned latent and need not decode observations. Value-aware learning, articulated by MuZero and the value-equivalence literature (§0.7), supplies the target the model is trained against. TD-MPC2 keeps the planner of the first line, drops the observation decoder of the second, and adopts the value functional as the training signal of the third. The single empirical commitment that organizes the paper is that one fixed hyperparameter configuration should work across many tasks and many model sizes, with no per-task tuning and no rescaling of components as the network grows.

0.8.1 Components and joint loss

The agent learns five components jointly. An encoder h maps an observation s_t to a normalized latent $z_t = h(s_t)$. A latent dynamics model d predicts the next latent, $\hat{z}_{t+1} = d(z_t, a_t)$. A reward model R predicts the immediate reward from the latent and action, $\hat{r}_t = R(z_t, a_t)$. A terminal action-value function Q supplies a bootstrap beyond the planning horizon, $\hat{q}_t = Q(z_t, a_t)$. A policy prior p supplies an amortized action proposal $\hat{a}_t = p(z_t)$ used both to warm-start the planner and to seed the bootstrap targets. The joint loss is the sum of a latent consistency term that pushes the predicted next latent to agree with the encoded next observation, $\|d(z_t, a_t) - \text{sg}(h(s_{t+1}))\|^2$ under a stop-gradient on the target encoding, a cross-entropy loss on the reward prediction, a cross-entropy loss on the temporal-difference targets for Q , and a behavioral cloning term that regresses p

⁹Voelcker et al. (2025) revisit this family and observe that the standard sampled losses, including the canonical MuZero objective, are uncalibrated surrogate losses whose minima do not in general coincide with the minima of the underlying target loss on the value function; value-equivalence is a property of an in-distribution policy and value class, and a model trained purely against value targets has no incentive to remain accurate once the policy drifts. The paper proposes a calibrated variant that restores the correctness property in expectation and leaves the broader question of value-aware accuracy under policy drift open.

¹⁰The same move appears in the operations-research line on decision-focused learning, in which a predictor is trained with a loss aligned to the downstream decision quality rather than to a statistical scoring rule on the forecast itself (Donti et al., 2017). The inner solver there is a convex program rather than a Bellman backup, but the conceptual move, to align the loss with what the decision-maker actually needs and to treat likelihood as the wrong default whenever the model class is misspecified, is shared.

on the actions selected by the planner. Notably absent is an observation reconstruction loss; the model is never asked to decode pixels or proprioceptive features and is trained only against quantities the value calculation actually consumes. This is the architectural commitment that distinguishes TD-MPC2 from the Dreamer line (§0.5), in which the world model carries a reconstruction term throughout.

0.8.2 MPPI planning with value bootstrap

Action selection at each environment step uses Model Predictive Path Integral control, a sampling-based optimizer over Gaussian action sequences (Hansen et al., 2024). The planner maintains a sequence of Gaussian distributions $(\mathcal{N}(\mu_h, \sigma_h^2))_{h=t}^{t+H-1}$ over the next H actions, draws a population of candidate sequences, rolls each one forward through the learned latent dynamics, scores the imagined returns, and refits (μ_h, σ_h) to a softmax-weighted average of the top-scoring sequences. The scoring rule is the finite-horizon return augmented with a terminal value bootstrap,

$$\mu^*, \sigma^* = \arg \max_{(\mu, \sigma)} \mathbb{E}_{(a_t, \dots, a_{t+H}) \sim \mathcal{N}(\mu, \sigma^2)} \left[\gamma^H Q(z_{t+H}, a_{t+H}) + \sum_{h=t}^{H-1} \gamma^{h-t} R(z_h, a_h) \right],$$

where the trajectory expectation is taken under the learned latent dynamics d rolled from the current encoded state $z_t = h(s_t)$. The terminal Q extends the planner’s effective horizon past the finite window H , so the agent does not need to roll long open-loop sequences through the learned model to capture long-horizon credit. This is the architectural reason TD-MPC2 sidesteps the long-rollout compounding error that limits PETS (§0.6), which has no terminal value and must extend H to capture distant reward. After the planner converges, the agent executes only $a_t \sim \mathcal{N}(\mu_t^*, (\sigma_t^*)^2)$ and re-plans at the next step. The amortized policy p seeds the next planning call and supplies the action used in the Q -bootstrap.

0.8.3 Empirical headline and scaling

The paper evaluates TD-MPC2 across 104 continuous-control tasks spanning four benchmarks, DeepMind Control, Meta-World, ManiSkill2, and MyoSuite, under a single fixed hyperparameter configuration with no per-task tuning. The task suite includes action dimensionalities up to 39, sparse rewards, multi-object manipulation, musculoskeletal motor

control, and locomotion on dog and humanoid embodiments. Across all four benchmarks TD-MPC2 outperforms SAC, the original TD-MPC, and DreamerV3 at comparable parameter counts. The contribution is not that any single task is solved better than ever before, but that one configuration solves all of them, which is the property the field had previously assigned to model-free baselines under heavy per-task tuning.

The scaling experiment trains a single multi-task agent on 80 tasks drawn from DeepMind Control and Meta-World and varies the world model size from 1 million parameters to 317 million. The normalized score moves from 16.0 at 1M parameters to 70.6 at 317M, roughly log-linear in parameter count over the two and a half orders of magnitude examined, with no observed saturation at the upper end. This is the first model-based reinforcement learning paper to report a clean monotone scaling curve over this range. The same hyperparameters are used at every model size; no learning rate, batch size, or capacity-specific schedule is retuned. The empirical claim of TD-MPC2 is that an MPC-shaped model-based agent, built from the components developed in the three earlier lines, scales with parameters in the same way that supervised and self-supervised models do.

0.9 Dual Simulation: Cobweb and Fishery

The chapter compares seven learning paradigms on two single-agent economic decision problems. The first is a cobweb model with adjustment cost, a self-referential environment in which the agent’s own action moves the contemporaneous price. The second is a logistic-growth fishery, an exogenous environment in which the agent’s action depletes a renewable stock but does not feed back through expectations. The seven paradigms are an oracle that knows the true parameters, a precautionary constant-rule baseline that does not learn, recursive least squares adaptive learning (Marcet and Sargent, 1989), the genetic algorithm of Arifovic (1994), tabular Q-learning (Watkins and Dayan, 1992), a model-based learner that estimates the structural coefficients by least squares and plans through a discounted Bellman equation (closed-form linear-quadratic on the cobweb, grid-based dynamic programming on the fishery), and a seventh paradigm that differs across panels: a simplified model-based REINFORCE learner in the spirit of Janner et al. (2019)’s MBPO¹¹, with bootstrap-ensemble dynamics, branched rollouts, and a linear

¹¹We label this learner MB-LG-REINFORCE (model-based linear-Gaussian REINFORCE) rather than MBPO to flag three departures from Janner et al. (2019). The dynamics ensemble is a bootstrap of linear-Gaussian regressions rather than dropout-regularized neural networks; the policy is a two-parameter Gaussian rather than

policy trained by REINFORCE on the cobweb, replaced on the fishery by a myopic open-access agent that maximizes the per-period profit and ignores stock dynamics, included to surface the textbook bioeconomic-collapse comparison that the cobweb’s no-learning constant rule does not exercise. The experiment is deliberately favorable to structured learners; the two environments have low-dimensional state, smooth payoffs, and stable parametric form, and the purpose is to locate the methods on an inductive-bias frontier rather than to rank them universally.

0.9.1 Cobweb with adjustment cost

The cobweb panel is designed to expose three things in turn: where each paradigm sits on the inductive-bias-vs-data trade-off, how parameter recovery rates differ across regimes, and where regret performance and policy quality come apart for methods that bake in correct functional form. The cobweb stability parameter b/c varies across regimes to modulate the curvature of the value function rather than to drive the kind of E-stability divergence [Marcet and Sargent \(1989\)](#) report for the multi-agent expectational cobweb, which does not arise in this single-agent monopoly variant.

Setup. The state is $s_t = (q_{t-1}, p_{t-1}) \in \mathbb{R}^2$ and the action is $q_t \in [0, 4]$. The price clears contemporaneously as $p_t = a - bq_t + \varepsilon_t$ with $\varepsilon_t \sim \mathcal{N}(0, \sigma^2)$, and the reward is $r_t = p_t q_t - (c/2)q_t^2 - (\phi/2)(q_t - q_{t-1})^2$, so the agent faces a linear demand, a quadratic production cost, and a quadratic adjustment cost. The sweep is $b/c \in \{0.5, 1.0, 2.0\}$ with $a = 4$, $c = 1$, $\phi = 0.2$, $\sigma = 0.1$, $\gamma = 0.95$, $T = 500$, and twenty independent seeds. The oracle policy is the affine feedback rule $q_t^* = K_0 + K_q q_{t-1}$ recovered by quadratic-form fixed-point iteration on the Bellman equation, and cumulative regret is computed per seed against the oracle’s realized return on the same noise sequence.

Models. Seven paradigms share the budget. The oracle knows (a, b, c, ϕ) and applies the closed-form optimal feedback rule. The constant-rule baseline plays $q_t = 1.4$ at every step regardless of state, providing the absolute no-learning floor. Recursive least squares of [Marcet and Sargent \(1989\)](#) estimates $(\hat{a}, \hat{b})$ from observed prices, treats (c, ϕ) as

a SAC actor with a separate critic and entropy term; and the rollout-disagreement reweighting that MBPO uses to compose model rollouts with real transitions is omitted. The branched-rollout outer loop with REINFORCE updates from the learned model is preserved. The simplification is appropriate for a linear-Gaussian cobweb but should not be read as a head-to-head with Janner’s algorithm.

known, and re-solves the linear-quadratic planner with point estimates each period.¹² The genetic algorithm of Arifovic (1994) maintains a population of binary-encoded production rules, evolves them under fitness-proportional selection on the running mean of realized observed profit, single-point crossover, bit-flip mutation, and two-elite preservation, and plays the population’s incumbent rule; the original Arifovic 1994 election operator scored hypothetical offspring against their parents using the true demand and cost parameters, and is omitted here so the agent uses only realized observed profit. Tabular Q-learning of Watkins and Dayan (1992) discretizes the state-action space onto a $20 \times 20 \times 25$ grid and applies ε -greedy temporal-difference updates with $\alpha = 0.1$ and ε decaying linearly from 0.3 to 0.01. Two model-based learners share the same parameter-estimation step but differ in how they plan. The model-based LQ learner exploits the linear-Gaussian structure: it solves the closed-form linear-quadratic Bellman equation by Riccati iteration on current point estimates of $(\hat{a}, \hat{b}, \hat{c}, \hat{\phi})$ each step and acts under the resulting affine feedback rule with Gaussian exploration noise. The MB-LG-REINFORCE learner, a simplified MBPO variant in the spirit of Janner et al. (2019), maintains a bootstrap ensemble of five linear-Gaussian demand models, parameterizes the policy as $q = K_0 + K_q q_{t-1}$, and at each real step samples ten rollouts of horizon five from buffer-uniform initial states under random ensemble members, accumulating discounted returns from the learned reward model and updating (K_0, K_q) by REINFORCE with a moving-average baseline.

Results. Table 2 reports cumulative regret at $T = 500$ for each paradigm and regime; Figure 3 shows the corresponding regret trajectories. Paradigms are presented in rank order by mean regret across regimes. Recursive least squares attains regret of about five units, indistinguishable across regimes within standard error, because it has the correct functional form for the price map and known cost parameters and need only estimate the two demand coefficients. The model-based LQ learner trails by an order of magnitude with regret rising from about twelve to forty-three units as the regime moves from stable

¹²The asymmetric structural prior across learners is deliberate but mixes two factors in the regret comparison. Recursive least squares is given (c, ϕ) , while the two model-based learners (model-based LQ and MB-LG-REINFORCE) must learn all four of (a, b, c, ϕ) . Its regret advantage over the model-based LQ learner therefore reflects both a tighter functional-form prior and the saved estimation budget for the cost coefficients, and the two cannot be cleanly separated within the present design. A fourth panel that re-runs recursive least squares without the known-cost prior would disentangle the two and is left for follow-up work; the qualitative ordering of paradigms on the inductive-bias frontier is robust to the asymmetry, since the model-based LQ learner pays a transient information cost on $(\hat{c}, \hat{\phi})$ that vanishes within the first hundred environment steps under the present parameters.

to unstable, reflecting the larger curvature it must resolve in the unstable region and the additional cost coefficients it must learn alongside the demand map. The genetic algorithm of Arifovic (1994) without the election operator is a further order of magnitude behind, with regret of ninety to three hundred units, reflecting the cost of gradient-free population search over a binary chromosome with no parametric knowledge of demand or cost. The constant rule shifts with the regime in a way that reflects the position of the fixed action relative to the regime’s optimum and is included as the absolute no-learning floor, not as a learning competitor. The MB-LG-REINFORCE learner sits next, with regret that varies more than two orders of magnitude across regimes (six hundred and fifty in stable, one hundred and twelve in borderline, forty-nine in unstable); the variance reflects REINFORCE’s sensitivity to the curvature of the return surface, which is flat in the stable regime where small differences in (K_0, K_q) matter little for return and sharp in the unstable regime where the policy gradient carries usable signal. Tabular Q-learning finishes near a thousand units of regret in every regime, the empirical signature of model-free tabular methods in continuous problems under a tight environment budget. The take-away is that on this cobweb, under the present hyperparameters, branched-rollout REINFORCE underperforms closed-form planning across all three regimes, with the gap widest in the stable regime where the return surface is flattest for the REINFORCE gradient estimator. The two methods share the same parameter-estimation step; the difference is what the planner does with the point estimates.

Regret captures the integrated cost of learning, not the asymptotic quality of the recovered model or policy. Figure 4 plots the parameter trajectories $\hat{a}_t$ and $\hat{b}_t$ for recursive least squares, the model-based LQ learner, and the MB-LG-REINFORCE ensemble mean against the true values, and Table 3 reports the final $|\hat{\theta} - \theta|$ error per paradigm and regime. All three methods recover the demand coefficients to within four percent of the truth in every regime; the two model-based learners additionally recover the cost coefficients (c, ϕ) to within machine precision¹³ because the reward signal pins them down once the residual $r_t - p_t q_t$ is regressed on $(q_t^2, (q_t - q_{t-1})^2)$. Recovery is fastest in the unstable regime because the curvature of the demand map sharpens the information content of each observation; this is the same observation that explains why the regret values themselves are similar

¹³The exact recovery reflects noiseless reward in this simulation: given observed (p_t, q_t, q_{t-1}) , the equation $r_t = p_t q_t - (c/2)q_t^2 - (\phi/2)(q_t - q_{t-1})^2$ is exactly solvable for (c, ϕ) from any two tuples that span the cost feature space. With additive reward noise $\sigma_r > 0$, recovery would degrade at rate $O(\sigma_r/\sqrt{N})$.

across regimes even though the absolute return at the oracle’s policy is lower in the unstable regime.

Figure 5 reports the distance from each learner’s noise-free action at a regime-specific reference state to the oracle’s action at the same state, on a log scale. The ordering is by inductive bias rather than recovery accuracy. The model-based LQ learner attains the smallest residual policy distance in every regime, two to three times smaller than recursive least squares in absolute terms, because by estimating (c, ϕ) it places its planner at the correct curvature of the value function. Recursive least squares is bounded below by the discrepancy between the prior on (c, ϕ) that fixes its planner and the realized environment, which is small here only because the prior was chosen to match. The genetic algorithm tracks a policy distance an order of magnitude larger, reflecting the discretization of the chromosome over the action space rather than incomplete recovery of any continuous parameter. The MB-LG-REINFORCE learner sits between the genetic algorithm and tabular Q-learning in the stable regime where REINFORCE has not converged its linear policy, and approaches the model-based LQ learner’s accuracy in the unstable regime where the gradient signal is strong; this regime dependence is the practical cost of the REINFORCE update relative to the analytical planner. Tabular Q-learning sits at order one, a direct consequence of the coarseness of the action grid (twenty-five points over a range of four units) compounded by the constant exploration noise injected into the rollout. The constant rule is flat by construction at a value that depends on how far the fixed action $q_{\text{fixed}} = 1.4$ lies from the regime’s optimum, and the value distinguishes how much of the no-learning floor is luck of the regime rather than any property of learning.

The verdict is that recursive least squares wins the regret comparison because its early decisions are nearly optimal under correct functional form and known cost parameters, while the model-based LQ learner wins the asymptotic policy-distance comparison because it ultimately fits more of the problem’s structure; the two methods occupy adjacent positions on the inductive-bias frontier, with recursive least squares paying a higher per-step penalty as long as its cost-parameter prior is correct and the model-based LQ learner paying a transient information cost in exchange for a tighter asymptote. The MB-LG-REINFORCE learner is the third position on this frontier: same model-estimation step as the LQ learner, but a slower policy update that pays off only when the closed-form structure is unavailable. On this cobweb the closed-form structure is available and the

comparison favors the LQ learner; in environments where it is not, the comparison would invert.

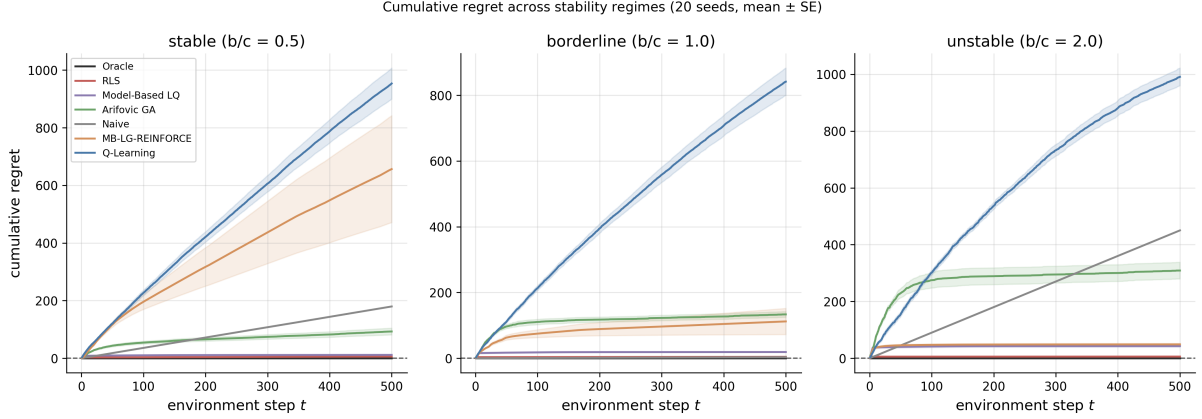


Figure 3: Cumulative regret across stability regimes for seven learning paradigms on the cobweb with adjustment cost. Lines are means and shaded bands are one standard error across twenty seeds. Panels correspond to $b/c \in \{0.5, 1.0, 2.0\}$. Oracle is the zero reference.

Table 2: Cumulative regret at $T = 500$ on the cobweb with adjustment cost, mean $\pm$ standard error across twenty seeds. Lower is better; Oracle is the reference.

Paradigm	Stable ($b/c = 0.5$)	Borderline ($b/c = 1.0$)	Unstable ($b/c = 2.0$)
Oracle	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
RLS	5.89 $\pm$ 1.04	4.38 $\pm$ 0.43	5.87 $\pm$ 0.18
Model-Based LQ	11.65 $\pm$ 0.93	18.96 $\pm$ 1.73	42.90 $\pm$ 3.75
Arifovic GA	92.89 $\pm$ 11.69	133.43 $\pm$ 8.65	308.88 $\pm$ 29.16
Naive	179.73 $\pm$ 0.34	3.37 $\pm$ 0.04	450.34 $\pm$ 0.34
MB-LG-REINFORCE	656.60 $\pm$ 185.41	112.06 $\pm$ 39.78	48.87 $\pm$ 3.47
Q-Learning	953.69 $\pm$ 53.79	841.50 $\pm$ 41.63	991.15 $\pm$ 31.19

0.9.2 Fishery with logistic growth

The fishery panel runs the structured-learner subset of the cobweb’s seven paradigms on a logistic-growth fishery, an exogenous environment that isolates the sample-efficiency story from the self-referential pricing of the cobweb. The reward structure is again linear-quadratic, but the transition dynamics are non-linear, so the oracle is a grid-based dynamic-programming solve rather than a closed-form linear-quadratic Riccati.

Setup. The state is the stock biomass $s_t \in [0, 1.5K]$ and the action is the harvest $h_t \in [0, \min(s_t, h_{\max})]$. The dynamics are $s_{t+1} = \max(0, s_t + rs_t(1 - s_t/K) - h_t + \varepsilon_t)$ with $\varepsilon_t \sim \mathcal{N}(0, \sigma^2)$, and the reward is $r_t = ph_t - (c/2)h_t^2$. Parameters are $r = 0.4$, $K = 10$,

Table 3: Final parameter recovery error $|\hat{\theta} - \theta|$ at $t = T$, mean $\pm$ standard error across twenty seeds. Cells marked — indicate parameters the paradigm does not estimate (recursive least squares assumes c and ϕ known); both model-based learners estimate all four.

Paradigm	Regime	$ \hat{a} - a $	$ \hat{b} - b $	$ \hat{c} - c $	$ \hat{\phi} - \phi $
RLS	stable	0.037 ± 0.035	0.015 ± 0.018	—	—
RLS	borderline	0.019 ± 0.026	0.011 ± 0.020	—	—
RLS	unstable	0.004 ± 0.016	0.003 ± 0.019	—	—
Model-Based LQ	stable	0.029 ± 0.019	0.013 ± 0.009	0.000 ± 0.000	0.000 ± 0.000
Model-Based LQ	borderline	0.004 ± 0.011	0.002 ± 0.008	0.000 ± 0.000	0.000 ± 0.000
Model-Based LQ	unstable	0.001 ± 0.007	0.003 ± 0.008	0.000 ± 0.000	0.000 ± 0.000
MB-LG-REINFORCE	stable	0.000 ± 0.005	0.000 ± 0.002	0.000 ± 0.000	0.000 ± 0.000
MB-LG-REINFORCE	borderline	0.005 ± 0.006	0.003 ± 0.004	0.000 ± 0.000	0.000 ± 0.000
MB-LG-REINFORCE	unstable	0.000 ± 0.005	0.002 ± 0.006	0.000 ± 0.000	0.000 ± 0.000

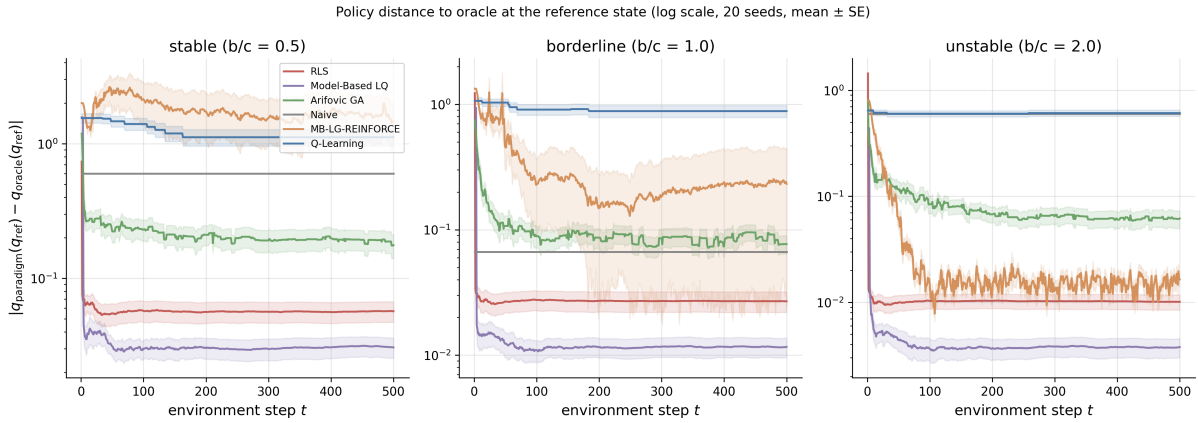


Figure 5: Policy distance to the oracle at a regime-specific reference state, log scale. The reference state is the static optimum quantity for each regime. Lines are means and bands one standard error across twenty seeds. Oracle is identically zero by construction and is omitted.

$h_t^{\text{MY}} = p/c = 10$, large enough to drive the stock to zero on the first step under the present parameters. The precautionary constant-rule baseline plays $h = 0.5$ at every step, a steady-state guess that lies between zero and the analytic MSY and that serves as a no-learning floor that does not collapse the fishery. Recursive least squares estimates $(\hat{r}, \hat{K})$ from a linear-in-parameters regression of $\Delta s_t + h_t$ on $(s_t, -s_t^2)$, plans by re-solving the DP with point estimates every twenty-five observations, and applies the resulting greedy policy with known cost parameters (p, c) . The model-based DP learner estimates $(\hat{r}, \hat{K}, \hat{p}, \hat{c})$ jointly by least squares, refits the DP every twenty-five observations, and acts with Gaussian exploration noise that decays over the episode; the planner is grid-based dynamic programming rather than the linear-quadratic Riccati of the cobweb sibling,

because the logistic transition is non-linear. Tabular Q-learning discretizes (s, h) onto a 30×21 grid with $\alpha = 0.1$ and ε decaying from 0.3 to 0.01. The genetic algorithm of Arifovic (1994) maintains thirty binary-encoded constant harvest rules and evolves them under fitness-proportional selection and a partial election operator with known cost parameters.¹⁴ The action support for the gradient-free and tabular learners (Q-Learning, the genetic algorithm, and the model-based DP learner’s exploration clip) is bounded above by $h_{\max} = 1.5 \cdot rK/4$ using the *true* r and K .¹⁵

Results. Table 4 reports cumulative regret at $T = 500$ for each paradigm in rank order; Figure 6 shows the regret trajectories. Recursive least squares and the model-based DP learner sit near the oracle (about thirteen and fifteen regret units respectively), tabular Q-learning recovers a usable policy at roughly two hundred and seventy-five units of regret, the precautionary constant rule lands at four hundred and forty-seven, the genetic algorithm finishes near seven hundred, and the myopic open-access agent at the bottom of the table accumulates seven hundred and fifty-three units of regret because it harvests the stock to zero on the first step and earns nothing thereafter; the textbook fishery-collapse tragedy is recovered. The fishery’s exogenous dynamics give the structured learners a cleaner identification problem than the cobweb because each (s_t, s_{t+1}) pair carries direct information about (r, K) that does not get confounded by the agent’s own action through expectations, and the consequence is that parameter recovery converges within the first hundred environment steps and the residual regret reflects exploration cost rather than mis-identification. Table 5 quantifies this: both structured learners recover r to within two percent and K to within three percent on average across the twenty seeds, with the model-based DP learner slightly tighter because it pools across observations in batch least squares rather than the recursive update of recursive least squares. The verdict mirrors the cobweb’s inductive-bias frontier with one twist: tabular Q-learning beats the precautionary constant rule, the genetic algorithm, and the myopic agent on this environment because the discretized state-action grid happens to be coarse enough to be

¹⁴The election operator scores child against parent on a static myopic profit $ph - (c/2)h^2$ evaluated at the most recently observed stock, rather than on a full discounted bioeconomic objective. This is faithful to the spirit of the Arifovic 1994 operator (which used the true demand and cost parameters to score offspring) but simplifies the comparison from a discounted-return rollout to a single-period reward; the chromosome encodes a constant harvest rule, so the simplification is mild.

¹⁵This bound constrains the action grid (and the chromosome decode range, and the exploration clip), not the learned policy. A model-free learner with no prior on r and K would need a separate mechanism to set its action support; we treat this as a fixed problem-specific scaffolding rather than a learned quantity.

tractable yet fine enough to track the optimal feedback rule’s monotone structure, while the genetic algorithm’s binary-encoded constant harvest is the wrong functional form for an environment where the optimal action varies smoothly with the stock and the population-search noise causes a non-trivial fraction of seeds to drift into harvest rates that themselves induce a collapse.

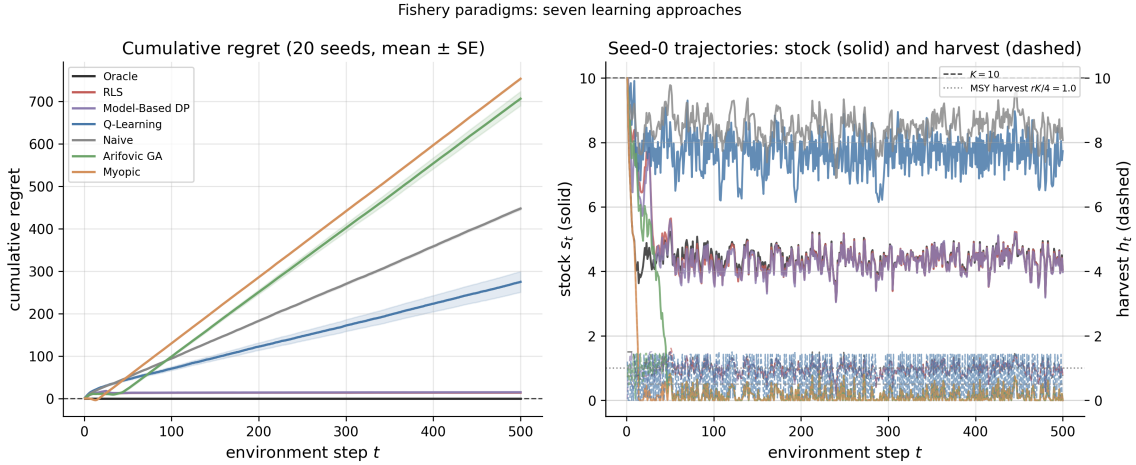


Figure 6: Left: cumulative regret on the logistic-growth fishery for seven paradigms (twenty seeds, mean and one-standard-error bands), presented in rank order by final regret. Right: seed-0 stock trajectories (solid) and harvest trajectories (dashed); the carrying capacity $K = 10$ and the maximum sustainable yield harvest $rK/4 = 1.0$ are shown as reference lines.

Table 4: Final cumulative regret at $T = 500$ on the fishery, mean $\pm$ standard error across twenty seeds. Lower is better; oracle is the zero reference. Paradigms are presented in rank order.

Paradigm	Final regret
Oracle	0.00 ± 0.00
RLS	13.67 ± 0.43
Model-Based DP	14.69 ± 0.73
Q-Learning	274.71 ± 24.36
Naive	447.35 ± 3.24
Arifovic GA	706.13 ± 16.65
Myopic	753.11 ± 1.75

0.10 Synthesis

The chapter has traced the model-based reinforcement learning line from two 1990 origins, Sutton’s Dyna and Schmidhuber’s controller-model architecture, through the deep revival in Ha and Schmidhuber (2018), the recurrent state-space model that became the workhorse of the Dreamer line, the value-aware question that runs from VAML through

Table 5: Parameter recovery on the fishery at $t = T$ for the two structured learners, mean $\pm$ standard error across twenty seeds. True values are $r = 0.4$ and $K = 10$. Recursive least squares assumes (p, c, σ) known and only estimates (r, K) ; the model-based DP learner additionally estimates (p, c) (not shown).

Paradigm	$\hat{r}$	$\hat{K}$	$ \hat{r} - r $	$ \hat{K} - K $
RLS	0.398 ± 0.002	10.034 ± 0.064	0.007	0.219
Model-Based DP	0.400 ± 0.002	9.965 ± 0.050	0.006	0.180

value-equivalence to MuZero, the modern Dyna in continuous control associated with the MBPO ensemble line, and the convergence point at TD-MPC2. The remainder of the section notes where model-based methods win, where they fail, where the surrounding boundary makes the model unnecessary, and what remains open.

0.10.1 Where model-based RL wins

Sample efficiency is the headline gain. Model-based methods outperform model-free baselines when environment interaction is scarce or expensive, including pixel-based continuous control in PlaNet and Dreamer, discrete-state planning in MuZero, and continuous control with sparse reward in MBPO and TD-MPC2. The learned model serves as a data amplifier through the Dyna planning step and as a representation amplifier, since the RSSM and TD-MPC2 latents transfer across tasks within their respective suites. When transferable representations or finite-data regimes are the binding constraint, model-based methods win.

0.10.2 Where model-based RL fails

The first failure mode is misspecification and the objective mismatch between training and use. [Lambert et al. \(2020\)](#) document that maximum-likelihood model fitting and reward optimization are decoupled objectives, so a model trained to high likelihood is not necessarily a model whose induced policy is high-return; their sweeps show likelihood-reward correlations as low as 0.07 on Half-Cheetah and adversarial perturbations that preserve likelihood while halving return. [Voelcker et al. \(2025\)](#) extend this point to MuZero and show that value-equivalent representations are systematically miscalibrated under distribution shift. The value-aware line (§0.7) is the partial answer, reformulating the training target around the value functional rather than the full distribution. A fully

principled treatment of off-distribution calibration remains open.

The second failure mode is compounding error and the data-coverage gaps that exploration must fill. [Asadi et al. \(2018b\)](#) give the local-operator Lipschitz bound for model-based RL, in which one-step prediction error compounds geometrically along the rollout at rate K_F , the Lipschitz constant of the learned dynamics, and the induced value-error bound on the planner stays finite when $\gamma K_F < 1$ while diverging as $\gamma K_F \rightarrow 1$. When the contraction condition fails, model-based planning diverges. [Talvitie \(2017\)](#) documents that retraining a single model on its own predictions can destabilize the model itself and proposes a self-correcting hallucinated-rollout scheme as one fix. [Pathak et al. \(2017\)](#) and [Sekar et al. \(2020\)](#) attack the same problem from the data side, with prediction-error-driven curiosity in the former and ensemble-disagreement-driven planning in the latter; both push the agent’s data distribution beyond what greedy exploitation alone produces, with measurable gains in regions of state space the agent would otherwise miss.

0.10.3 When the model is not needed

Not every sequential decision problem rewards the cost of learning a forecaster. When the decision reduces to a supervised prediction followed by a fixed downstream rule, the boundary studied by [Madeka et al. \(2022\)](#) in deep inventory management, a direct supervised reduction can match or beat a learned-world-model pipeline at much lower engineering cost. The boundary itself is closed-loop feedback. When the agent’s action does not feed back into the data-generating process, a supervised reduction suffices; when it does, the learned model becomes load-bearing because forecasts and decisions are then jointly determined. This is the boundary that places model-based reinforcement learning inside the sequential decision-making space and separates it from one-shot forecasting.

0.10.4 Open questions

The dual simulation in §0.9 maps where model-based reinforcement learning sits on the sample-efficiency frontier among the other paradigms in a single-agent cobweb with adjustment cost and a logistic-growth fishery. In the cobweb setting the Marcet-Sargent divergence-in-the-unstable-region story does not arise because the environment is self-referential through prices alone and not through expectations, and the more telling result is the order-of-magnitude separation between recursive least squares with correct func-

tional form, a model-based learner that must estimate the demand and the cost coefficients, a population-based gradient-free search, and a tabular model-free agent. Whether the divergence story would reappear in an expectational cobweb in which beliefs about future prices drive supply, and in particular how a learned-neural-network world model would behave in that regime, is one of the open questions the chapter does not yet settle. A second open question surfaces from the gap between the closed-form linear-quadratic planner and the branched-rollout REINFORCE planner on the cobweb panel: when the environment admits a closed-form planner, branched rollouts pay a variance cost that closed-form planning does not, and whether this gap survives on environments without analytical planners (precisely where deep MBPO is deployed in practice) is the empirical question the deep continuous-control benchmarks were designed to answer. The analogy to E-stability is suggestive rather than formal; the operators, fixed points, and misspecification objects differ, and a stability theory for model-based reinforcement learning in economic environments remains to be built. [Asadi et al. \(2018b\)](#) is the closest current analogue to a stability condition for the reinforcement-learning side, stating that value error grows geometrically along the rollout at rate K_F (the Lipschitz constant of the learned dynamics) and remains finite when $\gamma K_F < 1$, but the result is a per-rollout contraction bound on value error rather than a fixed-point characterization of the joint learning dynamics in the sense of [Marcet and Sargent \(1989\)](#).

Read against the monograph as a whole, this chapter is where the simulator dependence of reinforcement learning becomes explicit. The agent’s policy is only as good as the model it plans against, and the question of when a learned model is accurate enough for an economic policy counterfactual is the open one the monograph carries into its conclusion.

References

- Ioannis Antonoglou, Julian Schrittwieser, Sherjil Ozair, Thomas K. Hubert, and David Silver. Planning in stochastic environments with a learned model. In *ICLR*, 2022.
- Jasmina Arifovic. Genetic algorithm learning and the cobweb model. *Journal of Economic Dynamics and Control*, 18(1):3–28, 1994.
- Jasmina Arifovic. Genetic algorithms and inflationary economies. *Journal of Monetary Economics*, 36(1):219–243, 1995.
- Kavosh Asadi, Evan Cater, Dipendra Misra, and Michael L. Littman. Equivalence between Wasserstein and value-aware loss for model-based reinforcement learning. arXiv:1806.01265, 2018a.
- Kavosh Asadi, Dipendra Misra, and Michael L. Littman. Lipschitz continuity in model-based reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018b.
- Alex Ayoub, Zeyu Jia, Csaba Szepesvári, Mengdi Wang, and Lin F. Yang. Model-based reinforcement learning with value-targeted regression. In *ICML*, volume 119 of *PMLR*, pages 463–474, 2020.
- William A. Brock and Cars H. Hommes. A rational route to randomness. *Econometrica*, 65(5):1059–1095, 1997.
- George W. Brown. Iterative solution of games by fictitious play. In Tjalling C. Koopmans, editor, *Activity Analysis of Production and Allocation*, pages 374–376. John Wiley & Sons, 1951.
- Emilio Calvano, Giacomo Calzolari, Vincenzo Denicolò, and Sergio Pastorello. Artificial intelligence, algorithmic pricing, and collusion. *American Economic Review*, 110(10):3267–3297, 2020.
- Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *NeurIPS*, volume 31, 2018.

- Priya L. Donti, Brandon Amos, and J. Zico Kolter. Task-based end-to-end model learning in stochastic optimization. In *Advances in Neural Information Processing Systems*, volume 30, 2017. arXiv:1703.04529.
- Ignacio Esponda and Demian Pouzo. Berk-nash equilibrium: A framework for modeling agents with misspecified models. *Econometrica*, 84(3):1093–1130, 2016.
- George W. Evans and Seppo Honkapohja. *Learning and Expectations in Macroeconomics*. Princeton University Press, 2001.
- Amir-massoud Farahmand. Iterative value-aware model learning. In *NeurIPS*, volume 31, 2018.
- Amir-massoud Farahmand, André M. S. Barreto, and Daniel Nikovski. Value-aware loss function for model-based reinforcement learning. In *AISTATS*, volume 54 of *PMLR*, pages 1486–1494, 2017.
- Drew Fudenberg and David M. Kreps. Learning mixed equilibria. *Games and Economic Behavior*, 5(3):320–367, 1993.
- Drew Fudenberg and David K. Levine. *The Theory of Learning in Games*. MIT Press, Cambridge, MA, 1998.
- Christopher Grimm, André Barreto, Satinder Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *NeurIPS*, volume 33, 2020.
- Christopher Grimm, André Barreto, Gregory Farquhar, David Silver, and Satinder Singh. Proper value equivalence. In *NeurIPS*, volume 34, 2021.
- David Ha and Jürgen Schmidhuber. World models. In *Advances in Neural Information Processing Systems*, volume 31, 2018. arXiv:1803.10122.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.
- Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *ICLR*, 2020.

- Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. In *International Conference on Learning Representations (ICLR)*, 2021.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 2025. arXiv:2301.04104.
- Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024.
- Josef Hofbauer and William H. Sandholm. On the global convergence of stochastic fictitious play. *Econometrica*, 70(6):2265–2294, 2002.
- Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- Per Krusell and Anthony A. Smith. Income and wealth heterogeneity in the macroeconomy. *Journal of Political Economy*, 106(5):867–896, 1998. doi: 10.1086/250034.
- Thanard Kurutach, Ignasi Clavera, Yan Duan, Aviv Tamar, and Pieter Abbeel. Model-ensemble trust-region policy optimization. In *International Conference on Learning Representations (ICLR)*, 2018.
- Nathan Lambert, Brandon Amos, Omry Yadan, and Roberto Calandra. Objective mismatch in model-based reinforcement learning. In *Proceedings of the 2nd Conference on Learning for Dynamics and Control (L4DC)*, 2020.
- Dhruv Madeka, Kari Torkkola, Carson Eisenach, Anna Luo, Dean P. Foster, and Sham M. Kakade. Deep inventory management. arXiv:2210.03137, 2022.
- Albert Marcet and Thomas J. Sargent. Convergence of least squares learning mechanisms in self-referential linear stochastic models. *Journal of Economic Theory*, 48(2):337–368, 1989.
- Ramon Marimon, Ellen McGrattan, and Thomas J. Sargent. Money as a medium of exchange in an economy with artificially intelligent agents. *Journal of Economic Dynamics and Control*, 14(2):329–373, 1990.

- Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.
- Jürgen Schmidhuber. Making the world differentiable: On using self-supervised fully recurrent neural networks for dynamic reinforcement learning and planning in non-stationary environments. Technical Report FKI-126-90 (revised 11/90), Institut für Informatik, Technische Universität München, 1990.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020. doi: 10.1038/s41586-020-03051-4.
- Ramanan Sekar, Oleh Rybkin, Kostas Daniilidis, Pieter Abbeel, Danijar Hafner, and Deepak Pathak. Planning to explore via self-supervised world models. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.
- Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Proceedings of the Seventh International Conference on Machine Learning*, pages 216–224. Morgan Kaufmann, 1990.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, 2nd edition, 2018.
- Erik Talvitie. Self-correcting models for model-based reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, 2017.
- Claas Voelcker, Anastasiia Pedan, Arash Ahmadian, Romina Abachi, Igor Gilitschenski, and Amir-massoud Farahmand. Calibrated value-aware model learning with probabilistic environment models. In *ICML*, volume 267 of *PMLR*, pages 61745–61768, 2025.
- Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4): 279–292, 1992.
- H. Peyton Young. *Strategic Learning and its Limits*. Oxford University Press, Oxford, 2004.